7-4 · MinIO·Qdrant 연동 · 실습 1.5H

MinIO → 임베딩 → Qdrant 적재 파이프라인 실습

문서를 넣어 두면 알아서 검색 가능해지는 상태로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① kb.py 의 ingest() 로 순회 → 자르기 → 임베딩 → upsert 를 잇는다

② 재실행해도 포인트 수가 늘지 않는 것을 assert 로 확인한다

③ 검색 결과의 source 로 MinIO 의 원본 전체를 다시 읽어 온다

④ 줄어든 문서의 옛 조각을 재현하고 재적재 · 재색인으로 막는다

임베딩은 rag_env.py 의 get_emb() — 7-3 과 같은 모델, 같은 차원

실습 흐름

일곱 단계 — kb.py 하나를 만들고, 깨뜨리고, 고친다

오늘 실습 파일(kb_*.py)은 모두 kb.py 의 부품을 불러 씁니다

1. kb.py · 적재	2. 재실행	3. 일부러 실패	4. 역참조	5~7. 구멍 · 고침
세 문서 count=10	10 → 10 assert	uuid4 3 → 6	source 로 원본 읽기	6 → 6 → 1 재색인 10

1단계 · 부품

ingest() — 버킷의 .txt 를 훑어 lecture_kb 에 넣는다

list_objects	read_text	split_text	embed_documents	upsert
.txt 만 recursive=True	minio_env.py finally 로 반환	chunk_size 300 overlap 30	조각을 배치로 한 번에	id = uuid5 (source#i)

payload = {text, source: minio://raw-docs/..., chunk_index} · chunk 300 · 30 은 7-2 에서 정한 값

kb.py 를 만드는 상자는 출력이 없다 — emb = get_emb() · 컬렉션 lecture_kb · COSINE · 차원은 embed_query 로 잰다

1단계 · 적재

python kb_ingest.py — 세 문서, 열 조각

출력 줄	조각
minio://raw-docs/manual/guide.txt	1 chunks
.../notes/long_notes.txt	6 chunks
.../notes/notes.txt	3 chunks
적재: · 총 포인트:	10 · count=10

문서별 건수의 합과 count 가 같아야 한다 — 실측 · Ollama 에서 약 1분 반

2·3단계 — 일부러 실패시키기

멱등 확인, 그리고 uuid4 로 깨뜨려 본다

uuid5 — python kb_idem.py

같은 문서는 같은 자리를 덮어쓴다

- before = client.count(COLLECTION).count
- ingest() 한 번 더 → after
- 10 → 10 · 멱등 확인(실측)

VS

uuid4 — python kb_bad.py

실습용 컬렉션 idempotency_test 에서

- PointStruct(id=str(uuid.uuid4()), …)
- notes.txt 두 번 → count=3 다음 count=6
- 마지막 줄에서 delete_collection(TMP)

4단계

검색 결과에서 원본을 역참조한다 — 카드함에서 창고로

1. **query_points(..., limit=1) — 「작업 순서를 관리하는 도구는?」**
   「2장 오케스트레이션 / Airflow는 작업 간의 순서와…」(실측)

2. **출처: minio://raw-docs/notes/notes.txt**
   partition("minio://") → partition("/") 로 bucket, key

3. **read_text(bucket, key)**
   원본 전체가 「1장 데이터 파이프라인 개요」부터 찍힌다

카드 → 창고

source 로

원본 전체

> 조각은 일부만 들고 있어도 source 주소로 창고의 원본 전체를 꺼낸다

5단계 — 일부러 실패시키기

문서가 줄어들 때의 구멍을 재현한다

1. **count_of(SRC) — source 필터로 조각 수**
   변경 전: 6

2. **fput_object(BUCKET, "notes/long_notes.txt", "short.txt")**
   같은 키에 한 줄짜리 글을 덮어쓴다

3. **ingest(prefix="notes/long_notes.txt")**
   → 1 chunks · 변경 후: 6 ← 줄어들지 않았다(실측)

남은 것

1~5번

옛 조각

> **새 글은 0번 자리만 덮어썼다 — 검색하면 이미 지워진 내용이 나온다**

6·7단계

재적재로 고치고, 컬렉션째 다시 색인한다

reingest() — python kb_reingest.py

그 source 의 조각을 모두 지우고 다시 넣기

- client.delete(..., FilterSelector(filter=...))
- return ingest(prefix=object_name)
- 재적재: 1 · 최종: 1 ← 줄어들었다(실측)

VS

전체 재색인 — python kb_rebuild.py

원본을 되돌리고 컬렉션째 새로

- fput_object(..., "long_notes.txt") 로 원본 복구
- delete_collection → ingest()
- 1 · 6 · 3 조각 · 재색인: 10 · count=10

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. kb_ingest.py — 1 · 6 · 3 chunks, 적재: 10 · count=10
2. kb_idem.py — 10 → 10 과 멱등 확인
3. kb_bad.py — count=3 다음 count=6
4. kb_backref.py — 출처 minio://raw-docs/notes/notes.txt
5. kb_shrink.py 는 6 그대로 → kb_reingest.py 뒤 1
6. kb_rebuild.py — 재색인: 10 · 7-5 는 이 lecture_kb 를 쓴다

7-4 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

MinIO · Qdrant 코드를 한 표로 정리하고, 7-5 Hybrid Search 를 위한 준비를 확인합니다

- ✓ **MinIO · Qdrant 코드 요약표**
- ✓ **qdrant · rag-minio 둘 다 Up**
- ✓ **체크리스트와 자주 겪는 오류 네 가지**
- ✓ **7-5 준비: pip install rank-bm25**

가져갈 것

lecture_kb 에 10개 포인트